

EVALUACION	Obligatorio	GRUPO	Todos	FECHA	24/08/2026
MATERIA	Programacion 3				
CARRERA	AP/ATI				
CONDICIONES	Puntos: 30 Defensa obligatoria Entrega 12/11				

Descripción del problema

La plataforma de lectura interactiva "**Escoge tu propia aventura**" nos solicita desarrollar un sistema que permita gestionar historias interactivas: historias compuestas por capítulos encadenados, donde cada capítulo intermedio ofrece al lector varias opciones que lo llevan a otros capítulos, hasta alcanzar alguno de los posibles finales.

Existen dos roles de usuarios: **Administradores** y **Lectores**. De ellos se conoce su nombre completo, email, y un nombre de usuario y contraseña de mínimo 8 caracteres que debe incluir mayúsculas, minúsculas, números y caracteres especiales (al menos uno de cada uno), los que utilizan para loguearse en el sistema.

Categorías

Las historias se agrupan en categorías (ej.: Terror, Fantasía, Ciencia Ficción, Aventura). De cada categoría se conoce un identificador y un nombre.

Historias

De cada historia se conoce un identificador numérico, un título, una sinopsis, la o las categorías a la que pertenece, y el capítulo por el cual comienza la lectura (capítulo inicial). Una historia tiene además un estado: **BORRADOR** o **PUBLICADA**. Solo las historias publicadas pueden ser leídas por los usuarios. Una historia solo podrá pasar a estado PUBLICADA si el sistema valida que su estructura es correcta (ver reglas de negocio más abajo).

Capítulos

Son la entidad base del contenido de una historia. De cada sección se conoce un identificador numérico, un título y un texto. Existen dos tipos de capítulos:

Intermedios: Tienen asociada una lista de opciones. Cada opción tiene un texto (ej.: "Abrir la puerta", "Huir por la ventana") y un capítulo destino al cual conduce esa elección.

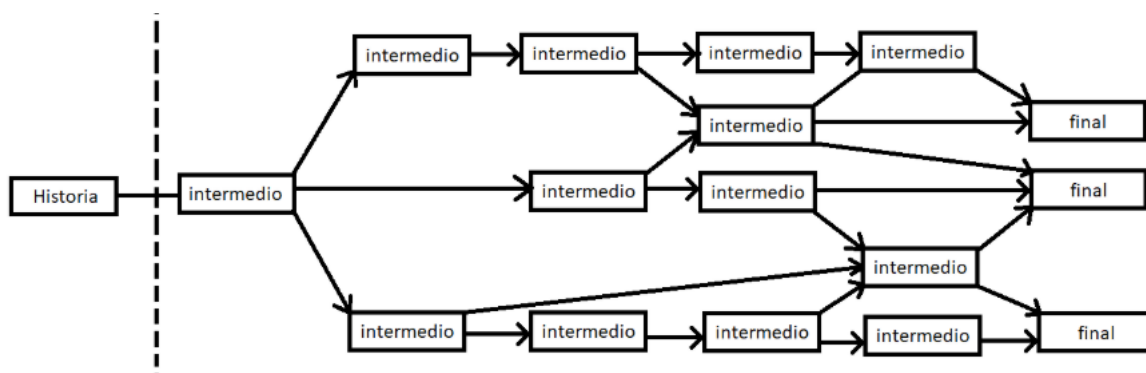
Finales: No tienen opciones. Llevan la cuenta de la cantidad de veces que los lectores llegaron a ese final.

Reglas de negocio para las historias:

- Las historias tienen un único capítulo inicial y no puede ser de tipo final.
- Mientras se están creando tendrán estado BORRADOR y al publicarlas para permitir su lectura tendrán estado PUBLICADA.
- Los capítulos se encadenan a través de opciones que llevan a otros capítulos desde el capítulo actual.
- Solamente se podrá llegar a otro capítulo a continuación de capítulos de tipo intermedio. Los finales no tienen capítulos siguientes.
- Todos los capítulos de las historias (excepto los finales) deberán tener al menos una opción y todas las opciones deberán llevar a otro capítulo (intermedio o final).
- Los capítulos finales solamente tendrán título y texto y un contador de las veces que se alcanzó ese final durante las lecturas de la historia.
- No se pueden compartir capítulos intermedios ni finales entre diferentes historias.

Estas reglas podrían sufrir modificaciones o aparecer nuevas reglas de negocio en el futuro. Si esto ocurre, se informará oportunamente.

Posible grafo de historia con sus capítulos (no se representan las opciones)



Lectura de una historia

Cuando un usuario comienza a leer una historia, el sistema registra una lectura, comenzando en el capítulo inicial. En cada capítulo intermedio el usuario elige una de las opciones disponibles y avanza al capítulo destino, hasta llegar a un Final. El sistema debe guardar, para cada lectura: el usuario, la historia, la fecha de inicio, la fecha de finalización (si terminó), y el final alcanzado (si corresponde). Un mismo usuario puede leer la misma historia más de una vez (por ejemplo, para descubrir otros finales) y en cada caso se creará una nueva lectura.

Estilos de lectura

El sistema debe permitir que cada usuario elija entre **dos estilos visuales distintos** de presentación para leer las historias (por ejemplo, un estilo claro y uno oscuro). Este tema no se verá en el curso: deberán investigarlo por su cuenta, apoyándose en alguna herramienta de inteligencia artificial generativa para implementarlo, y deberán incluir como parte de la documentación del obligatorio la investigación realizada y la forma en que decidieron implementarlo.

Arquitectura de la solución

El sistema tendrá dos soluciones .NET independientes, ambas desarrolladas en Visual Studio 2026, utilizando C# como lenguaje de programación y .NET versión 10:

1. Aplicación de Administración

La aplicación de Administración estará compuesta por una única solución, con Web MVC, arquitectura Limpia, SOLID y DDD, implementando las funcionalidades de administración (RF01 a RF05).

En este caso, no será necesario exponer ni consumir una Web API.

2. Aplicación de Lectura

La aplicación de lectura de historias estará separada de la de administración y estará compuesta por:

- a. Una Web API, responsable de exponer mediante endpoints RESTful y utilizando la misma arquitectura y diseño de la aplicación de Administración, los

requerimientos RF01 y RF06 a RF11.

- b. Una aplicación Web MVC cliente, encargada exclusivamente de la presentación y de consumir la Web API del punto anterior mediante HttpClient teniendo en cuenta el RF12.

La autenticación de la aplicación de Lectura se resolverá mediante JWT. El login devolverá un token que la aplicación MVC deberá almacenar y enviar posteriormente en las solicitudes a los endpoints protegidos mediante el encabezado Authorization.

Requerimientos funcionales

Aplicación de Administración (MVC con Arquitectura Limpia, sin Web API)

- **RF01 – Login de administradores** A la aplicación de administración podrán ingresar los usuarios con rol Administrador mediante sus credenciales. La autenticación se resuelve dentro de la misma aplicación MVC. Se implementará el Login y Logout.
- **RF02 – Alta de usuarios – Rol Administrador** Se ingresan los datos básicos del usuario y se le asigna uno de los roles existentes (Administrador o Usuario).
- **RF03 – Gestión de historias – Rol Administrador** CRUD de historias.
Al momento de crear una historia, esta ya se creará con un capítulo inicial. Este capítulo no tendrá opciones, solamente título y texto, y la historia quedará en estado BORRADOR.

Para dar de baja una historia, el sistema debe controlar que no tenga lecturas asociadas, de lo contrario, mostrará un mensaje descriptivo.

La edición solamente permitirá cambiar título, sinopsis, categorías y el estado. El capítulo inicial se podrá modificar siempre y cuando no se hayan agregado ya otros capítulos a la historia. Cuando una historia se pasa al estado PUBLICADA, deberá controlarse que todos los capítulos tengan al menos una opción y que todas las opciones lleven a otro capítulo (intermedio o final).

- **RF04 – Alta de capítulos intermedios y finales – Rol Administrador** Permite agregar los capítulos (intermedios o finales) de una historia.

Al agregar capítulos, se indica la historia para la cual se agregan, su tipo (intermedio o final) y se indica cuál es el capítulo que le antecede. Se ingresará el título y el texto del capítulo, así como el texto de la opción respectiva. Como resultado, al capítulo antecedente se le creará una opción con ese texto, que tendrá como destino el nuevo capítulo creado. Solamente se podrán agregar capítulos a continuación de capítulos de tipo intermedio (no finales).

Los capítulos finales solamente tendrán título y texto y un contador de las veces que se alcanzó ese final durante las lecturas de la historia.

No se pueden compartir capítulos intermedios ni finales entre diferentes historias.

- **RF05 – Auditoría de historias – AUTOMÁTICO SISTEMA / Rol Administrador** Las acciones de creación, edición y publicación de historias y capítulos deberán ser auditable: el sistema registrará la acción realizada, la fecha y el usuario administrador que la realizó. Un administrador podrá consultar esta información filtrando por historia o por administrador.

Web API (Arquitectura Limpia) + Aplicación MVC cliente (HttpClient + JWT)

- **RF06 – Login de usuarios** A esta aplicación podrán ingresar tanto Administradores (para acceder a las métricas expuestas por la API) como Usuarios (para leer historias), El login se resuelve mediante **token JWT** consumiendo un endpoint de la Web API. La aplicación MVC cliente deberá conservar ese token y enviarlo en cada llamado subsiguiente a la Web API para acceder a los endpoints protegidos. Se implementará el Login y Logout.
- **RF07 – Lectura de historia – Rol Usuario** El sistema mostrará el listado de historias publicadas (filtrable por categoría). El usuario selecciona una, comienza la lectura desde el capítulo inicial y, en cada capítulo intermedio, elige una opción para avanzar, hasta llegar a un final. El sistema debe registrar el final alcanzado cuando esto ocurra. Una vez alcanzado un posible final de la historia, para intentar un nuevo camino deberá iniciar una nueva lectura.
- **RF08 – Historial de lecturas – Rol Usuario** Se mostrará un listado de las lecturas del usuario logueado, realizadas en un mes y año dados, indicando la historia, si fue completada y, en ese caso, el final alcanzado. Si no existen lecturas que mostrar, deberá informarlo mediante un mensaje descriptivo.

- **RF09 – Ranking de historias más leídas – Rol Administrador** Listado de historias ordenado por cantidad de lecturas completadas, de mayor a menor. Las historias nunca leídas no deben aparecer en el listado.
- **RF10 – Final más alcanzado por historia – Rol Administrador** Se selecciona una historia de una lista y el sistema muestra, para cada uno de sus finales, la cantidad de veces que fue alcanzado, indicando cuál es el más frecuente.
- **RF11 – Usuario con más historias leídas – Rol Administrador** Ranking de usuarios según la cantidad de historias distintas que completaron (leyeron hasta un final), ordenados de mayor a menor cantidad.
- **RF12 – Selección de estilo de lectura – Rol Usuario** El usuario podrá elegir, entre los dos estilos visuales disponibles (por ejemplo, fondo claro y fondo oscuro). Este requerimiento no se verá en el curso: deberá investigarse por su cuenta, apoyándose en IA generativa, y documentarse en el informe.

Reglas generales

- A menos que se especifique lo contrario, todos los datos son requeridos.
- Todas las entidades deberán identificarse mediante una propiedad Id de tipo entero (int). El Id será **autonumérico y secuencial**, generado exclusivamente por la **base de datos** al momento de persistir la entidad.
- Las entidades del dominio deberán poder validarse a sí mismas mediante la implementación de la interfaz **IValidable**.
- Cuando corresponda se deberán implementar **Value Objects**.
- Las validaciones, errores, etc. se manejarán a través de **excepciones propias**.
- En la aplicación Web, cada vez que se produzca un error en las ABM, o cuando no se obtengan resultados para las solicitudes de información, se deberá **informar convenientemente al usuario**. En el caso de las Web Api se deberán retornar los **códigos de estado adecuados según la guía REST**.
- Cuando sea necesaria información para la operativa y su registro no sea explícitamente solicitado en los requerimientos, será **cargada manualmente** en la base de datos.
- Las consultas se realizarán a través de **LINQ to Entities exclusivamente**.
- En el caso de utilizar código generado mediante inteligencia artificial generativa, deberá estar en condiciones de **explicarlo en la defensa**.

NOTA: Durante las altas y modificaciones se deberán realizar todas las validaciones explicitadas en la descripción de las entidades, en los requerimientos detallados, o todas aquellas que sean relevantes y **de sentido común** a pesar de no estar explicitadas en el presente documento.

Se pide

1. Implementar todas las funcionalidades descritas anteriormente y generar la documentación que se detalla más abajo.
2. Incluir la documentación de la WebApi (con Swagger).
3. La API se podrá probar mediante Swagger o Postman.
4. Realizar precarga de datos (historias, capítulos, opciones y finales) mediante la ayuda de IA generativa. **Se deberán incluir en la documentación la captura y el link a la secuencia de los prompts utilizados.** Los datos deberán ser fácilmente identificables, coherentes con la realidad planteada y aptos para resolver todas las búsquedas/métricas solicitadas (por ejemplo, historias con múltiples finales alcanzados con distinta frecuencia).
5. Esos datos deberán figurar en el script de base de datos que se entrega (inserts).

Requerimientos no funcionales

- Este proyecto se realizará usando bibliotecas de clase, proyecto Web MVC y proyecto Web Api de .NET 10, C# como lenguaje de programación, Entity Framework 10 para el acceso a la base de datos y LINQ como lenguaje de consultas. Se preferirá LINQ con sintaxis de método.
- Se deberá organizar el backend y el frontend siguiendo las recomendaciones de Arquitectura Limpia (Clean Architecture), y DDD (Domain Driven Design) y deberán aplicarse los principios SOLID vistos en el curso.

Pre-entregas (en Aulas)

Tarea	Fecha de entrega
Entrega 1 - Diagrama con el diseño y arquitecturas elegidas. Solución .NET con proyectos/carpetas/clases (sin código). Planificación del proyecto	Jueves 10/9
Entrega 2 - RF01, RF02, RF03	Jueves 24/9
Entrega 3 - RF06, RF07, RF08	Jueves 15/10
Entrega 4 - RF10, RF11, RF12	Jueves 21/10
ENTREGA DE OBLIGATORIO en gestión	Jueves 12/11

Entrega final (en gestión.ort.edu.uy)

Se subirá a gestión un único archivo comprimido que contenga:

A. Implementación Archivo comprimido en formato .zip, .rar o .7z con la solución funcionando completa.

B. Precarga de datos en la base, de acuerdo con los requerimientos mencionados como un script SQL.

C. PDF con la documentación siguiente:

- Carátula con nombre, número de estudiante y foto de los integrantes del grupo.
- Tabla de contenido con la página en que se ubica cada ítem.
- Diagrama de casos de uso de todo el sistema.
- Descripción narrativa de los siguientes casos de uso:
 - Lectura de historia (RF06)
 - Historial de lecturas (RF07)
- Diseño de casos de prueba para el ranking de historias más leídas (RF08).
- Diagrama de clases (UML) completo de la capa de lógica de negocio, y para las demás capas solamente las clases involucradas en la funcionalidad Lectura de historia (RF06). Se excluyen las Views de MVC.

g. Documentar la generación de datos de prueba creados usando IA generativa.

h. Documentar la investigación realizada para implementar la selección de estilos (RF05).

D. Archivo Astah con los diagramas anteriormente solicitados.

E. Scripts SQL con los juegos de datos de prueba (mínimo 10 registros por tabla, esquemas + inserts, incluyendo precargas y datos que surgen del uso del sistema).

F. Despliegue en SOME de la aplicación Web MVC, la Web API y la base de datos SQL Server, con hipervínculo en la documentación.

G. Link al repositorio del obligatorio en GitHub, dentro de la organización correspondiente, con el formato de nombre acordado (grupo + número/s de estudiante/s).

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la entrega online de obligatorios son:

Ingresá al sistema de Gestión.

En el menú, seleccioná el ítem “Evaluaciones” y la instancia de evaluación correspondiente, que figura bajo el título “Inscripto”.

Para iniciar la entrega hacé clic en el ícono: C:\Users\shirl\AppData\Local\Microsoft\Windows\INetCache\Content.MSO\4FCEF9D0.tmp

Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en “Agregar”. El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en “Crear equipo”.

Cualquier integrante podrá:

- Modificar la integración del equipo.
- Subir el archivo de la entrega.

Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en “Subir” para iniciar la entrega. Cada equipo (hasta 2 estudiantes) debe entregar un único archivo en formato zip o rar (los documentos de texto deben ser pdf, y deben ir dentro del zip o rar). El archivo a subir debe tener un tamaño máximo de 40mb

Cuando el archivo quede subido, se mostrará el nombre generado por el sistema (1), el tamaño y la fecha en que fue subido.

El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.

Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.

La hora tope para subir el archivo será las 21:00 del día fijado para la entrega.

La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).

Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora o Coordinación adjunta antes de las 20:00hs. del día de la entrega, a través de los mails, alamon@ort.edu.uy, martinez_i@ort.edu.uy o fernandez_ma@ort.edu.uy; o telefónicamente al 29021505 - int 1156 u 1138 6 u 1138